

Es crucial que los alumnos comprendan los fundamentos de HTML y CSS antes de sumergirse en la generación de vistas desde el backend con .NET. **Material de Estudio: Desarrollo Web Básico (HTML/CSS)**

Profesor: Jorge Izaguirre **Materia:** Programación .NET **Curso:** 6to Año - TÉCNICO EN PROGRAMACIÓN

Parte 1: Entendiendo la Web y Construyendo la Estructura con HTML

(1.1) ¿Cómo Funciona la Web? Clientes, Servidores y HTTP

Cuando navegas por internet usando un navegador (como Chrome, Firefox, Edge), estás participando en una comunicación constante basada en el modelo **Cliente-Servidor**.

- **Modelo Cliente-Servidor:**

- *Cliente:* Es tu navegador web. Su trabajo es *solicitar* recursos (páginas web, imágenes, videos, etc.) y *mostrar* la respuesta al usuario.
- *Servidor Web:* Es una computadora potente (o un conjunto de ellas) que está siempre conectada a internet, esperando solicitudes de los clientes. Su trabajo es *almacenar* los recursos del sitio web y *enviar* respuestas a las solicitudes que recibe.

- **HTTP (HyperText Transfer Protocol):** Es el *protocolo*, el conjunto de reglas o el "idioma" que utilizan el cliente y el servidor para comunicarse a través de internet. La comunicación básica sigue un ciclo de Petición-Respuesta:

1. **Petición HTTP (Request):** El cliente (navegador) envía un mensaje al servidor solicitando un recurso. Esta petición incluye:
 - *Método:* La acción que se desea realizar. Los más comunes son:
 - *GET:* Solicita obtener un recurso (ej: cargar una página web, ver una imagen). Es el método más común al navegar. Los datos viajan en la URL (visible).
 - *POST:* Envía datos al servidor para que los procese (ej: enviar un formulario de registro, subir un archivo). Los datos viajan en el cuerpo de la petición (no visible en la URL).
 - *URL:* La dirección única del recurso solicitado (ej: <https://www.ejemplo.com/productos/zapatillas>).
 - *Cabeceras (Headers):* Información adicional (tipo de navegador, cookies, etc.).
2. **Respuesta HTTP (Response):** El servidor procesa la petición y envía un mensaje de vuelta al cliente. Esta respuesta incluye:
 - *Código de Estado:* Un número de 3 dígitos que indica el resultado de la petición. Algunos comunes:
 - *200 OK:* ¡Todo salió bien! El recurso solicitado se envía en el cuerpo.
 - *404 Not Found:* El servidor no pudo encontrar el recurso solicitado en esa URL.
 - *403 Forbidden:* No tienes permiso para acceder a ese recurso.
 - *500 Internal Server Error:* Hubo un error en el servidor al procesar la petición.
 - *Cabeceras (Headers):* Información adicional sobre la respuesta (tipo de contenido, cookies para establecer, etc.).
 - *Cuerpo (Body):* (Opcional) El contenido del recurso solicitado (ej: el código HTML de la página, los datos de la imagen).

- **URL (Uniform Resource Locator):** La dirección completa que identifica un recurso en la web. Ej: <https://www.google.com/search?q=escuela+tecnica+3> (protocolo [https](https://), dominio www.google.com, ruta [/search](https://www.google.com/search), parámetros [?q=...](https://www.google.com/search?q=...)).

(1.2) Introducción a HTML: El Esqueleto de la Web

HTML (HyperText Markup Language) no es un lenguaje de programación, sino un **lenguaje de marcado**. Su única función es describir y organizar la **estructura y el contenido** de una página web. Le dice al navegador *qué* es cada pieza de información (un título, un párrafo, una lista, una imagen), pero **no** le dice *cómo* debe verse visualmente (ese es el trabajo de CSS).

- **Elementos:** Son las unidades básicas de construcción de una página HTML (ej: un encabezado, un párrafo, una imagen).
- **Etiquetas (Tags):** Son las marcas que usamos en el código para definir los elementos. La mayoría vienen en pares: una etiqueta de apertura (`<nombre>`) y una etiqueta de cierre (`</nombre>`). El contenido del elemento va entre ellas.
 - Ejemplo: `<p>Este es el contenido del párrafo.</p>`
- **Etiquetas Auto-cerradas (Void Elements):** Algunas etiquetas no tienen contenido y se cierran solas.
 - Ejemplo: `<br>` (salto de línea), `<img>` (imagen), `<hr>` (línea horizontal), `<input>` (campo de formulario). En HTML5, la barra final (`/>`) es opcional, pero a veces se usa por claridad (`<br />`).
- **Atributos:** Proporcionan información adicional o configuración para un elemento. Se escriben dentro de la etiqueta de apertura, con el formato `nombre="valor"`.
 - Ejemplo: `<a href="https://www.google.com">Ir a Google</a>` (el atributo `href` especifica la URL del enlace).
 - Ejemplo: `` (el atributo `src` indica la ruta de la imagen, `alt` proporciona texto alternativo).

(1.3) Estructura Básica de un Documento HTML5

Todo archivo HTML debe seguir una estructura fundamental para que los navegadores lo entiendan correctamente:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mi Primera Página Web</title>
</head> <body>
  <h1>El Encabezado Más Importante</h1>
  <p>Un párrafo de texto con un <a href="otra_pagina.html">enlace interno</a>.
</p>
  

</body> </html>
```

- ¿Cómo crearlo? Simplemente abre un editor de texto (como VS Code, Notepad++, o incluso el Bloc de Notas), escribe esta estructura, guárdala con un nombre que termine en `.html` (ej: `index.html`), y luego abre ese archivo con tu navegador web.

(1.4) Etiquetas HTML Comunes para Estructurar Contenido

- **Encabezados (<h1> a <h6>):** Definen títulos y subtítulos. <h1> es el más importante (generalmente uno por página), <h6> el menos importante. Ayudan a estructurar jerárquicamente el contenido y son importantes para SEO.

```
<h1>Título Principal</h1>
<h2>Subtítulo de Sección</h2>
<h3>Sub-subtítulo</h3>
```

- **Párrafos (<p>):** Para bloques de texto. El navegador añade espacio antes y después por defecto.

```
<p>Este es un párrafo. Puede contener mucho texto.</p>
<p>Este es otro párrafo.</p>
```

- **Enlaces (<a>):** Crean hipervínculos. El atributo href es obligatorio e indica la URL destino.

```
<a href="https://www.google.com">Buscar en Google (Enlace externo)</a>
<a href="contacto.html">Ir a la página de Contacto (Enlace interno)</a>
<a href="#seccion_importante">Ir a una sección de esta página</a>
<a href="mailto:info@example.com">Enviar Email</a>
```

- **Imágenes ():** Muestran una imagen. Atributos clave:
 - **src:** La ruta (URL) al archivo de imagen (puede ser local relativa: `imagenes/foto.jpg` o absoluta: `https://sitio.com/logo.png`). **Obligatorio.**
 - **alt:** Texto alternativo que describe la imagen. **Fundamental** para accesibilidad (lectores de pantalla) y SEO, y se muestra si la imagen no carga. **Obligatorio.**
 - **width, height:** Opcionales para especificar tamaño (aunque es mejor hacerlo con CSS).

```

```

- **Listas:** Para agrupar ítems relacionados.
 - **No Ordenadas ():** Muestran ítems con viñetas (bullets).

```
<ul>
  <li>Primer ítem</li>
  <li>Segundo ítem</li>
  <li>Otro ítem</li>
</ul>
```

- **Ordenadas ():** Muestran ítems numerados o con letras.

```

<ol>
  <li>Paso 1</li>
  <li>Paso 2</li>
  <li>Paso 3</li>
</ol>

```

- Dentro de `<ul>` y `<ol>`, solo pueden ir directamente elementos `<li>` (List Item).
- **Tablas (`<table>`):** Para mostrar datos organizados en filas y columnas (datos tabulares). **No deben usarse para maquetar el layout general de la página** (para eso usamos CSS).
 - `<table>`: Envuelve toda la tabla.
 - `<thead>`: (Opcional pero recomendado) Define la sección de encabezado de la tabla.
 - `<tbody>`: (Opcional pero recomendado) Define el cuerpo principal de la tabla.
 - `<tfoot>`: (Opcional) Define la sección de pie de tabla.
 - `<tr>`: Define una fila (Table Row).
 - `<th>`: Define una celda de encabezado (Table Header). Usualmente dentro de `<thead>`. El texto se muestra en negrita y centrado por defecto. El atributo `scope` (`scope="col"` o `scope="row"`) mejora la accesibilidad.
 - `<td>`: Define una celda de datos (Table Data). Usualmente dentro de `<tbody>`.

```

<table>
  <thead>
    <tr>
      <th scope="col">Nombre</th>
      <th scope="col">Apellido</th>
      <th scope="col">Nota</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Ana</td>
      <td>Sol</td>
      <td>8</td>
    </tr>
    <tr>
      <td>Luis</td>
      <td>Mar</td>
      <td>6</td>
    </tr>
  </tbody>
</table>

```

Parte 2: Semántica, Formularios, Estilo con CSS y Layout Básico

(2.1) HTML Semántico: Dando Significado a la Estructura

Podríamos construir toda una página usando solo etiquetas `<div>` (divisores genéricos) y `<span>` (contenedores en línea genéricos) y luego darle estilo con CSS. Pero esto es una **mala práctica**. HTML5 introdujo etiquetas **semánticas** que describen el *propósito* o *significado* de la sección que contienen.

- *¿Por qué usar HTML Semántico?*
 - **Accesibilidad:** Ayuda a los lectores de pantalla y otras tecnologías asistivas a entender la estructura de la página y navegarla mejor.
 - **SEO (Search Engine Optimization):** Ayuda a los motores de búsqueda (Google, Bing) a comprender de qué trata cada parte de tu contenido, lo que puede mejorar tu posicionamiento.
 - **Claridad del Código:** Hace que tu código HTML sea mucho más fácil de leer y entender para otros desarrolladores (¡y para tu yo futuro!).
- **Etiquetas Semánticas Comunes para Layout:**
 - **<header>**: Contiene la introducción o cabecera principal del sitio o de una sección. Usualmente incluye el logo, el título principal (**<h1>**), y a veces la navegación principal.
 - **<nav>**: Contiene los enlaces de navegación principales del sitio (el menú).
 - **<main>**: Representa el contenido **principal y único** de la página. Debe haber solo un **<main>** por página (directamente dentro de **<body>**). Excluye cabeceras, pies de página, barras laterales y navegación repetitiva.
 - **<article>**: Representa un contenido completo o autocontenido que podría distribuirse de forma independiente (ej: un post de blog, un artículo de noticias, un comentario de usuario, un widget). Puede haber varios **<article>** dentro de **<main>**.
 - **<section>**: Representa una sección temática genérica dentro de un documento. Usualmente tiene un encabezado (**<h2>**, **<h3>**, etc.) que la identifica. Sirve para agrupar contenido relacionado dentro de **<main>** o **<article>**.
 - **<aside>**: Representa contenido que está relacionado tangencialmente con el contenido principal que lo rodea (una barra lateral con enlaces relacionados, publicidad, biografía del autor).
 - **<footer>**: Representa el pie de página del sitio o de una sección. Usualmente contiene información de copyright, enlaces a términos de servicio, información de contacto.

Ejemplo de Estructura Semántica:

```
<body>
  <header>
    <h1>Mi Blog Técnico</h1>
    <nav>
      <ul>
        <li><a href="/">Inicio</a></li>
        <li><a href="/categorias">Categorías</a></li>
        <li><a href="/acerca">Acerca de</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <article>
      <h2>Título del Artículo 1</h2>
      <p>Contenido del artículo...</p>
      <section>
        <h3>Comentarios</h3>
        <p>Comentario 1...</p>
      </section>
    </article>
  </main>
</body>
```

```

</article>

<article>
  <h2>Título del Artículo 2</h2>
  <p>Otro contenido...</p>
</article>
</main>

<aside>
  <h3>Archivo</h3>
  <ul>...</ul>
</aside>

<footer>
  <p>&copy; 2025 Jorge Izaguirre. Todos los derechos reservados.</p>
</footer>
</body>

```

(2.2) Formularios HTML: Interactuando con el Usuario

Los formularios son esenciales para que los usuarios puedan enviar información a nuestro sitio web (ej: iniciar sesión, registrarse, buscar, contactar).

- **Elemento `<form>`:** Envuelve todos los controles del formulario. Atributos clave:
 - **`action`:** La URL del script en el *servidor* que recibirá y procesará los datos del formulario cuando se envíe. (En MVC, esto apuntará a una *Action* de un *Controller*).
 - **`method`:** El método HTTP a usar para enviar los datos.
 - **`GET`:** Los datos del formulario se añaden a la URL visible en la barra de direcciones (`?nombre=valor&otro=valor2`). Útil para formularios de búsqueda o cuando no se envía información sensible. Tiene limitaciones de longitud.
 - **`POST`:** Los datos se envían en el cuerpo de la petición HTTP (no visible en la URL). Es el método preferido para enviar formularios con información sensible (contraseñas), datos extensos, o que modifiquen datos en el servidor (registros, etc.).
- **Controles de Formulario (`<input>`, `<textarea>`, `<select>`, `<button>`):** Son los elementos que permiten al usuario ingresar datos.
 - **`<input>`:** El más versátil. Su comportamiento cambia según el atributo **`type`**.
 - **`type="text"`:** Campo de texto normal.
 - **`type="password"`:** Campo de contraseña (oculta los caracteres).
 - **`type="email"`:** Campo para email (navegador puede validar formato básico).
 - **`type="number"`:** Campo para números (puede tener flechas +/-).
 - **`type="date"`:** Selector de fecha.
 - **`type="checkbox"`:** Casilla de verificación (múltiples se pueden seleccionar si tienen diferente *name*, o agrupar con el mismo *name*). Se envía un valor si está marcada.
 - **`type="radio"`:** Botón de opción. Varios radios con el **mismo** atributo *name* forman un grupo donde solo uno puede ser seleccionado. Requieren atributo *value*.
 - **`type="file"`:** Permite al usuario seleccionar un archivo para subir.
 - **`type="submit"`:** Botón que, al ser presionado, envía el formulario (ejecuta el *action* con el *method* especificado). El atributo *value* define el texto del botón.
 - **`type="reset"`:** Botón que limpia todos los campos del formulario a sus valores iniciales.

- `type="button"`: Botón genérico sin acción por defecto, usualmente usado con JavaScript.
- **<label>**: Etiqueta de texto asociada a un control. **Fundamental para accesibilidad y usabilidad.** Se asocia usando el atributo `for` en el `<label>` que debe coincidir con el atributo `id` del control (`<input>`, `<textarea>`, etc.). Al hacer clic en la etiqueta, el foco se pone en el control asociado.

```
<label for="nombreUsuario">Nombre de Usuario:</label>
<input type="text" id="nombreUsuario" name="username">
```

- El atributo `name` en los controles es **crucial**, ya que es el nombre que se usará para identificar el dato cuando se envíe al servidor.
- **<textarea>**: Campo para ingresar texto multilínea.

```
<label for="mensaje">Mensaje:</label><br>
<textarea id="mensaje" name="user_message" rows="4" cols="50">
</textarea>
```

- **<select>**: Lista desplegable. Contiene elementos `<option>`.

```
<label for="provincia">Provincia:</label>
<select id="provincia" name="provincia_seleccionada">
  <option value="">Seleccione una...</option>
  <option value="BA">Buenos Aires</option>
  <option value="CBA">Córdoba</option>
  <option value="SF">Santa Fe</option>
</select>
```

El atributo `value` de la `<option>` seleccionada es lo que se envía al servidor.

- **<button>**: Alternativa más flexible a `<input type="submit/reset/button">`. Puede contener HTML dentro. Su comportamiento por defecto dentro de un `<form>` es `type="submit"`.

```
<button type="submit">Enviar Datos</button>
<button type="reset">Limpiar</button>
<button type="button">Botón Genérico</button>
```

(2.3) Introducción a CSS: Dando Estilo a la Web

CSS (Cascading Style Sheets) es el lenguaje que usamos para controlar **cómo se ven** los elementos HTML en el navegador. Define colores, fuentes, tamaños, espaciados, posicionamiento, etc.

- ¿Por qué separar HTML de CSS? **Separación de Intereses (Separation of Concerns):**
 - **HTML:** Se encarga de la **estructura** y el **significado** del contenido.
 - **CSS:** Se encarga de la **presentación** visual.

- *Ventajas:*

- **Mantenimiento:** Cambiar el diseño visual del sitio es mucho más fácil si solo necesitas editar los archivos CSS, sin tocar el HTML.
- **Reutilización:** Puedes aplicar los mismos estilos a muchas páginas simplemente enlazando el mismo archivo CSS.
- **Accesibilidad:** Un HTML bien estructurado y semántico es legible incluso sin CSS.
- **Rendimiento:** Los archivos CSS se pueden cachear por el navegador.

- **Formas de Aplicar CSS (Priorizando la Mejor Práctica):**

1. **Hoja de Estilos Externa (External Stylesheet): ¡La forma recomendada y estándar!**

- Creas un archivo de texto plano con extensión `.css` (ej: `estilos.css`).
- Dentro de este archivo, escribes todas tus reglas CSS.
- En la sección `<head>` de **cada** archivo HTML que quieras estilizar, enlazas la hoja de estilos usando la etiqueta `<link>`:

```
<head>
  <link rel="stylesheet" href="css/estilos.css">
</head>
```

2. *Hoja de Estilos Interna (Internal Stylesheet):*

- Escribes las reglas CSS dentro de una etiqueta `<style>` ubicada en el `<head>` del documento HTML.
- *Desventaja:* Los estilos solo aplican a *esa* página HTML específica. No es reutilizable fácilmente. Usar solo para estilos muy puntuales de una única página.

```
<head>
  <title>Página con Estilos Internos</title>
  <style>
    body { background-color: lightblue; }
    h1 { color: navy; }
  </style>
</head>
```

3. *Estilos en Línea (Inline Styles):*

- Aplicas estilos directamente a una etiqueta HTML usando el atributo `style`.
- Ejemplo: `<p style="color: red; font-weight: bold;">Texto importante.</p>`
- **¡Evitar en lo posible!** Mezcla presentación con estructura, es muy difícil de mantener, anula la reutilización y tiene alta especificidad (difícil de sobrescribir). Usar solo como último recurso para casos muy específicos.

(2.4) Sintaxis Básica de CSS y Selectores

CSS funciona mediante **Reglas**. Cada regla consta de un **Selector** y un **Bloque de Declaraciones** entre llaves `{}`. Cada declaración es un par **Propiedad: Valor**;

```
/* Esto es un comentario en CSS */

Selector {
  Propiedad1: Valor1;
  Propiedad2: Valor2;
  /* ... más propiedades ... */
}
```

- **Selectores:** Indican a qué elemento(s) HTML se aplicará la regla de estilo. Los básicos son:
 - **Selector de Tipo (o Etiqueta):** Aplica a todas las etiquetas de ese tipo.

```
p { /* Aplica a todos los párrafos <p> */
  color: darkslategray;
  line-height: 1.6; /* Espacio entre líneas */
}
h1 { /* Aplica a todos los <h1> */
  font-family: Arial, sans-serif;
  color: navy;
}
```

- **Selector de Clase:** Aplica a todos los elementos que tengan el atributo `class` con ese nombre específico. Se define con un punto (.) seguido del nombre de la clase (sin espacios, sensible a mayúsculas/minúsculas). **Es el selector más versátil y reutilizable.**

```
/* En CSS */
.destacado {
  background-color: yellow;
  font-weight: bold;
}
.boton {
  padding: 10px 15px;
  border: 1px solid #ccc;
  background-color: #eee;
  text-decoration: none;
  color: black;
  border-radius: 4px; /* Bordes redondeados */
}
.boton-primario {
  background-color: blue;
  color: white;
  border-color: darkblue;
}
```

```
<p>Texto normal, pero <span class="destacado">esto está
destacado</span>.</p>
```

```
<a href="#" class="boton">Botón Normal</a>
<a href="#" class="boton boton-primario">Botón Primario</a>
```

- **Selector de ID:** Aplica al **único** elemento en la página que tenga ese atributo `id` específico. Se define con una almohadilla (#) seguida del ID (sin espacios, sensible a mayúsculas/minúsculas). **El ID debe ser único en toda la página.**

```
/* En CSS */
#menu-principal {
    background-color: #333;
    padding: 10px;
}
#logo {
    width: 150px;
}
```

```
<nav id="menu-principal">...</nav>

```

- ¿Cuándo usar ID vs Clase? Usa **Clases** para estilos que podrías querer reutilizar en varios elementos (botones, alertas, destacados, etc.). Usa **IDs** para identificar elementos estructurales únicos en la página (la cabecera principal, el menú principal, el pie de página) o como anclas para enlaces internos (`#seccion`) o para manipulación específica con JavaScript (aunque las clases también sirven para JS). **Prioriza el uso de Clases.**
- **Cascada y Especificidad (Concepto):** CSS significa *Cascading Style Sheets* (Hojas de Estilo en Cascada). Esto significa que los estilos "caen" o se aplican en cascada. Si un elemento coincide con varios selectores, ¿cuál gana?
 - **Especificidad:** El navegador calcula un "peso" o especificidad para cada selector. Los selectores más específicos "pesan" más. En orden de mayor a menor especificidad (simplificado): Estilos Inline (`style="..."`) > Selectores de ID (`#id`) > Selectores de Clase (`.clase`), Atributo (`[type="text"]`) y Pseudo-clase (`:hover`) > Selectores de Tipo (`p`, `div`) y Pseudo-elemento (`::before`).
 - **Orden:** Si dos reglas tienen la *misma* especificidad, la que aparece **última** en el archivo CSS (o la última leída por el navegador) es la que se aplica.
 - **!important:** (¡Evitar si es posible!) Una declaración marcada con `!important` (ej: `color: red !important;`) sobrescribe casi cualquier otra regla, ignorando la especificidad. Dificulta mucho el mantenimiento y la depuración.

(2.5) Propiedades CSS Comunes (Primer Vistazo)

Hay cientos de propiedades CSS. Aquí algunas de las más usadas para empezar:

- **Color y Fondo:**
 - `color:` Color del texto. (Valores: nombres como *red*, *blue*; códigos hexadecimales como `#FF0000` (rojo), `#333` (gris oscuro); `rgb(255, 0, 0)`; `rgba(255, 0, 0, 0.5)` (con transparencia)).

- **background-color**: Color de fondo del elemento.
- **background-image**: `url('ruta/imagen.jpg');` Imagen de fondo.
- **Fuentes:**
 - **font-family**: Lista de fuentes a usar, separadas por coma (el navegador usa la primera que encuentre instalada). Terminar con una familia genérica (*sans-serif*, *serif*, *monospace*). Ej: **font-family: Arial, Helvetica, sans-serif;**.
 - **font-size**: Tamaño de la fuente (ej: **16px**, **1.2em**, **1rem**, **100%**).
 - **font-weight**: Grosor de la fuente (*normal*, *bold*, o números como 400, 700).
 - **font-style**: Estilo de la fuente (*normal*, *italic*, *oblique*).
- **Texto:**
 - **text-align**: Alineación horizontal (*left*, *right*, *center*, *justify*).
 - **text-decoration**: Subrayado, tachado, etc. (*none*, *underline*, *line-through*).
 - **line-height**: Altura de línea (espaciado vertical entre líneas de texto).
 - **letter-spacing**, **word-spacing**: Espaciado entre letras o palabras.
- **Dimensiones:**
 - **width**: Ancho del elemento.
 - **height**: Alto del elemento.
 - **max-width**, **min-width**, **max-height**, **min-height**: Límites para las dimensiones.
- **Bordes:**
 - **border**: Propiedad corta para definir todos los lados. Ej: **border: 2px dashed green;** (ancho, estilo, color).
 - Propiedades individuales: **border-width**, **border-style** (*solid*, *dotted*, *dashed*, *double*, etc.), **border-color**.
 - También por lado: **border-top**, **border-right**, **border-bottom**, **border-left**.
 - **border-radius**: Para crear esquinas redondeadas. Ej: **border-radius: 5px;**.
- **Espaciado (Modelo de Caja - Ver Siguierte Sección):**
 - **padding**: Relleno *interno* (entre el contenido y el borde).
 - **margin**: Margen *externo* (entre el borde y los elementos vecinos).
 - Se pueden definir los 4 lados: **padding: 10px 20px 10px 5px;** (arriba, derecha, abajo, izquierda) o **padding: 10px 20px;** (arriba/abajo, izquierda/derecha) o **padding: 10px;** (todos los lados). También individuales: **padding-top**, **margin-left**, etc.

(2.6) El Modelo de Caja CSS (Box Model)

Fundamental para entender el layout y el espaciado. Todo elemento HTML en la página es tratado por CSS como una **caja rectangular**. Esta caja tiene 4 capas:

1. **Content (Contenido)**: El área donde se muestra el texto, la imagen, etc. Su tamaño se define (a veces) por **width** y **height**.
 2. **Padding (Relleno)**: Un espacio transparente *alrededor* del contenido, pero *dentro* del borde. Separa el contenido del borde.
 3. **Border (Borde)**: Una línea que rodea el padding y el contenido. Tiene grosor, estilo y color.
 4. **Margin (Margen)**: Un espacio transparente *fuera* del borde. Separa esta caja de las cajas de los elementos vecinos. Los márgenes verticales entre elementos adyacentes a veces se "colapsan" (solo se aplica el mayor).
- **El Problema del box-sizing**: Por defecto (con **box-sizing: content-box;**), si defines **width: 200px; padding: 10px; border: 1px solid black;**, el ancho *total* ocupado por la caja en

pantalla será $200\text{px (content)} + 10\text{px (padding izq)} + 10\text{px (padding der)} + 1\text{px (border izq)} + 1\text{px (border der)} = 222\text{px}$. ¡Esto hace muy difícil calcular layouts precisos!

- **La Solución (`box-sizing: border-box`);** Es una propiedad CSS que cambia este comportamiento. Si la aplicas, el `width` y `height` que definas se aplicarán al tamaño total de la caja **incluyendo el padding y el borde**. El navegador ajustará automáticamente el espacio del *contenido* hacia adentro para que todo encaje. ¡Mucho más intuitivo!
 - **Práctica recomendada:** Aplicar esto a todos los elementos al principio de tu CSS:

```
*,
*::before,
*::after {
    box-sizing: border-box;
}
```

(El `*` es el selector universal, aplica a todo).

(2.7) Introducción a Flexbox para Layout

Durante años, maquetar páginas web (posicionar elementos) era complicado (se usaban `float`, `position`, tablas - ¡mal!). **Flexbox** es un módulo de layout moderno de CSS diseñado para distribuir espacio y alinear ítems en **una dimensión** (ya sea una fila o una columna).

- ¿Por qué usar Flexbox? Es potente, flexible y relativamente fácil de aprender para layouts unidimensionales comunes (barras de navegación, filas de tarjetas, centrado de elementos).
- **Conceptos Clave:**
 - **Contenedor Flex (Flex Container):** El elemento HTML padre al que le aplicas la propiedad `display: flex`; Esto "activa" Flexbox para sus hijos directos.
 - **Items Flex (Flex Items):** Los hijos directos del contenedor flex. Se convierten en ítems flexibles que pueden ser posicionados por el contenedor.
 - **Ejes:** Flexbox trabaja con dos ejes:
 - *Eje Principal (Main Axis):* La dirección en la que se colocan los ítems flex (por defecto es horizontal, como una fila). Definido por `flex-direction`.
 - *Eje Transversal (Cross Axis):* El eje perpendicular al principal (por defecto es vertical).
- **Propiedades Básicas (Aplicadas al Contenedor Flex):**
 - `display: flex`; ¡Lo primero! Activa Flexbox.
 - `flex-direction: row | row-reverse | column | column-reverse`; Define la dirección del eje principal. Por defecto es `row` (horizontal, izquierda a derecha). `column` los apila verticalmente.
 - `justify-content: flex-start | flex-end | center | space-between | space-around | space-evenly`; **Alinea** los ítems a lo largo del **eje principal**.
 - `flex-start`: Agrupa al inicio.
 - `flex-end`: Agrupa al final.
 - `center`: Agrupa al centro.
 - `space-between`: Espacio uniforme *entre* ítems (el primero pega al inicio, el último al final).
 - `space-around`: Espacio uniforme *alrededor* de cada ítem (más espacio en los bordes que entre ítems).
 - `space-evenly`: Espacio uniforme *entre* ítems y entre ítems y los bordes.

- *align-items: stretch | flex-start | flex-end | center | baseline;* **Alinea** los ítems a lo largo del **eje transversal**.
 - *stretch*: (Por defecto) Estira los ítems para ocupar todo el espacio transversal.
 - *flex-start*: Alinea al inicio del eje transversal.
 - *flex-end*: Alinea al final del eje transversal.
 - *center*: Alinea al centro del eje transversal.

- **Ejemplo Simple: Barra de Navegación Horizontal:**

```
<nav class="menu">
  <a href="#">Inicio</a>
  <a href="#">Servicios</a>
  <a href="#">Contacto</a>
</nav>
```

```
.menu {
  display: flex; /* Activa flexbox para los <a> dentro */
  background-color: #333;
  padding: 10px;
  justify-content: space-around; /* Espacia los enlaces */
  align-items: center; /* Centra verticalmente si los enlaces tuvieran
diferente altura */
}

.menu a { /* Estilo para los enlaces dentro del menú */
  color: white;
  text-decoration: none;
  padding: 5px 10px;
}
```

Flexbox es mucho más potente (manejo de crecimiento/encogimiento de ítems con *flex-grow*, *flex-shrink*, *flex-basis*; envolver ítems con *flex-wrap*), pero esto es una introducción.

(2.8) Herramientas de Desarrollador del Navegador (DevTools)

Son tu **mejor amigo** para el desarrollo front-end. Todos los navegadores modernos las incluyen.

- ¿Cómo abrirlas? Generalmente presionando la tecla **F12**, o haciendo clic derecho en cualquier parte de la página y seleccionando "Inspeccionar" o "Inspect Element".
- **Funciones Clave para HTML/CSS:**
 - **Pestaña *Elements* (Chrome/Edge) o *Inspector* (Firefox):**
 - Muestra el **árbol DOM** (la estructura HTML tal como la interpreta el navegador, no necesariamente idéntica al código fuente si hay errores o manipulación con JS).
 - Puedes **seleccionar** elementos en el árbol (o haciendo clic en ellos en la página con la herramienta de selección) para verlos resaltados.
 - **Panel *Styles* (Estilos):** En un panel lateral, muestra **todas** las reglas CSS que aplican al elemento seleccionado, indicando qué archivo CSS y línea provienen, y cuáles están siendo sobrescritas (tachadas) debido a la cascada o especificidad.

- **¡Puedes editar CSS en vivo!** Puedes activar/desactivar checkboxes de propiedades, cambiar valores, añadir nuevas propiedades directamente en el panel *Styles* y ver los cambios reflejados en la página **en tiempo real**. ¡Ideal para experimentar y depurar estilos! (Los cambios son temporales, no se guardan en tu archivo `.css`).
- También muestra el **Modelo de Caja** visualmente para el elemento seleccionado, con sus dimensiones, padding, border y margin.
- *¿Por qué son indispensables?* Permiten ver exactamente qué HTML está renderizando el navegador y qué CSS se está aplicando (y por qué), facilitando enormemente la identificación y corrección de problemas de estructura y estilo. ¡Úsalas constantemente!

Conclusión

Hemos recorrido los fundamentos esenciales para construir la parte visible de la web. **HTML** nos da las herramientas para **estructurar el contenido y darle significado semántico**. **CSS** nos permite controlar la **aparición visual y la disposición (layout)** de ese contenido, separando la presentación de la estructura. Entender cómo funcionan el modelo **Cliente-Servidor** y **HTTP** nos da contexto. Dominar las **Herramientas de Desarrollador** es crucial para trabajar eficientemente. Con estas bases, estarás mucho mejor preparado para entender cómo frameworks como ASP.NET Core MVC generan dinámicamente el HTML y cómo puedes aplicar estilos CSS para crear interfaces web atractivas y funcionales.

Síntesis de una oración: Este material explica los conceptos básicos de la web (Cliente/Servidor, HTTP), cómo estructurar contenido semánticamente con HTML5 (incluyendo formularios y tablas) y cómo aplicar estilos y layout con CSS3 (selectores, propiedades, Box Model, Flexbox básico), utilizando las DevTools del navegador como herramienta fundamental.

Síntesis de 5 puntos importantes:

1. La web funciona bajo un modelo **Cliente-Servidor** comunicándose vía **HTTP** (Requests/Responses).
2. **HTML** define la **estructura semántica** del contenido mediante etiquetas (ej: `<h1>`, `<p>`, `<nav>`, `<form>`).
3. **CSS** controla la **presentación visual** (colores, fuentes, espaciado, layout) y debe separarse del HTML en archivos `.css` externos.
4. CSS aplica estilos mediante **Selectores** (tipo, `.clase`, `#id`) a elementos, considerando el **Box Model** (content, padding, border, margin) y la **Cascada/Especificidad**.
5. **Flexbox** es una herramienta CSS moderna para crear layouts unidimensionales flexibles, y las **DevTools (F12)** son esenciales para inspeccionar y depurar HTML/CSS.

37367